Stock Market Direction Prediction

Deep Industry-Level Machine Learning Project Report

Problem: Predict whether the next trading day's closing price will be higher than today's close.
Reference: NIFTY 50 (^NSEI), configurable to other OHLCV instruments.
Companion: Google Colab end-to-end notebook with 8 algorithms, graphs, walk-forward validation and new-data inference.

Prepared as a technical project document for classroom explanation, portfolio presentation, project submission and interview/viva discussion.

Important: Educational ML demonstration only. The model output is not investment advice and does not guarantee future market movement.

Table of Contents

1. Executive Summary
2. Business Context & Problem Definition
3. Objectives, Scope & Assumptions
4. End-to-End Architecture
5. Data Acquisition & Data Dictionary
6. Data Quality & Preprocessing
7. Exploratory Data Analysis
8. Technical Indicators & Feature Engineering
9. Target Engineering & Leakage Prevention
10. Chronological Validation Design
11. Baseline Model
12–19. Eight ML Algorithms
20. Evaluation Metrics
21. Model Comparison & Threshold Selection
22. Final Test Evaluation
23. Feature Importance
24. Walk-Forward Validation
25. New Unseen Data Inference
26. Model Persistence
27. Deployment Architecture
28. MLOps & Monitoring
29. Testing & Governance
30. Risks & Limitations
31. Future Enhancements
32. Result Template
33. Interview/Viva Questions
34. Conclusion

1. Executive Summary

This project implements an end-to-end supervised machine-learning system for next-day stock-market direction classification. Raw historical Open, High, Low, Close and Volume data is transformed into momentum, trend, volatility, volume, range and candle-based predictors. The project explicitly addresses the most common failure mode in financial ML—future-data leakage—by creating features only from information available at time t and preserving chronological order throughout model development.

2. Business Context & Problem Definition

A market participant or analytics team may want a probabilistic estimate of short-horizon direction to support research, dashboards or downstream decision systems. The technical question is deliberately narrow: using information known at the end of trading day t, can a model estimate whether Close(t+1) will be greater than Close(t)? This is converted into a binary classification task. The project focuses on ML engineering quality rather than claiming profitable trading performance.

3. Objectives, Scope & Assumptions

Primary objective: build a reproducible, leakage-aware classification pipeline and compare multiple algorithms on future chronological periods.
Secondary objectives: understand market data visually, engineer meaningful features, benchmark against a simple baseline, explain predictions through feature importance, support new-row inference and prepare the model for deployment.
Scope: daily OHLCV technical data. **Out of scope:** guaranteed returns, intraday execution, portfolio optimization, brokerage/tax modeling and personalized investment advice.

4. End-to-End Architecture

Data Source → Schema Check → Data Audit → EDA → Feature Engineering → Target Creation → Drop Unavailable Rolling Rows → Chronological Train/Validation/Test → Baseline → Eight Candidate Models → Validation Metrics → Threshold Analysis → Final Refit → Untouched Test → Permutation Importance → Walk-Forward Evaluation → New Raw OHLCV → Feature Pipeline → Probability → Saved Model/Metadata → API/Dashboard/Monitoring.

5. Data Acquisition & Data Dictionary

The Colab notebook uses **yfinance** to request daily historical data. The default symbol is **^NSEI** (NIFTY 50). The ticker and start date are parameters, so the same pipeline can be reused for another supported instrument after validating its data characteristics.

Column	Definition	Why needed
Date	Trading session date	Time ordering, splitting, calendar context
Open	First traded/quoted session price	Gap and candle-body features
High	Highest session price	True range and intraday range
Low	Lowest session price	True range and intraday range
Close	End-of-session close	Returns, indicators and prediction target
Volume	Session traded volume where available	Participation/activity features

> Industry note: a production system should document data vendor, timezone, corporate-action handling, symbol changes, missing sessions and revision policy. A model is only as trustworthy as its data lineage.

Core acquisition code

```
df = yf.download(TICKER, start=START_DATE,
                 auto_adjust=False, progress=False)
df = df.reset_index()
```

```
df = df[["Date","Open","High","Low","Close","Volume"]]
```

The code deliberately selects a stable schema instead of passing every vendor-returned column into the model.

6. Data Quality & Preprocessing

Before EDA or modeling, the notebook checks shape, dtypes, missing values, duplicate dates and descriptive statistics. Dates are sorted chronologically. Rolling indicators naturally create missing values during their warm-up windows; those rows are removed only after all features and the target are defined.

> **Why this matters:** blindly filling the first 200 rows of an SMA-200 feature would fabricate information. The safer choice is to require enough historical lookback and train only where the engineered feature is genuinely available.

```
print(df.info())
display(df.isna().sum())
print("Duplicate dates:", df["Date"].duplicated().sum())
display(df.describe().T)
```

7. Exploratory Data Analysis

EDA is used to understand behavior and detect modeling risks—not to prove that a visual pattern will continue. The project includes the following graph families.

Graph	What students should observe	Industry interpretation
Closing price	Long-term trend, drawdowns, regimes	Price level is non-stationary; avoid naive extrapolation
Daily return histogram	Center, tails, extreme moves	Returns can be noisy and heavy-tailed
Rolling volatility	Quiet vs turbulent periods	Model behavior may vary by regime
Volume trend	Participation changes	Volume scale and missing/zero values need review
SMA overlays	Short vs long trend	Relative features are often more stable than raw price
RSI	Momentum oscillator behavior	Thresholds are heuristics, not labels
MACD	Momentum/trend crossover	Useful feature, not guaranteed signal
Bollinger bands	Price relative to rolling band	Context for local price position
Target balance	UP vs NOT-UP frequency	Accuracy can mislead if imbalance exists
Correlation matrix	Linear redundancy	High correlation may indicate redundant features

8. Technical Indicators & Feature Engineering

Feature engineering converts raw OHLCV into information the model can learn from. Every feature is computed from present or past observations only.

8.1 Returns and Momentum

Formula: Return(k) = Close(t) / Close(t-k) – 1. The project uses 1, 2, 5, 10 and 20-day returns. These represent recent movement over multiple horizons. A positive value indicates price is above the earlier close; magnitude indicates relative change.

```
x["Return_1D"]  = x["Close"].pct_change(1)
x["Return_5D"]  = x["Close"].pct_change(5)
x["Return_20D"] = x["Close"].pct_change(20)
```

8.2 Simple Moving Averages

SMA is the arithmetic mean of recent closing prices. The project creates SMA 5, 10, 20, 50, 100 and 200. Instead of feeding only raw SMA values, it also creates **Close / SMA** – **1**, which expresses relative distance from trend.

```
for w in [5,10,20,50,100,200]:
    x[f"SMA_{w}"] = x["Close"].rolling(w).mean()
    x[f"Close_to_SMA_{w}"] = x["Close"] / x[f"SMA_{w}"] - 1
```

8.3 EMA and MACD

EMA assigns more weight to recent observations. MACD is EMA(12) – EMA(26); a 9-period EMA of MACD becomes the signal line, and MACD – Signal becomes the histogram. The model receives the numerical relationship rather than a hard-coded trading rule.

```
x["EMA_12"] = x["Close"].ewm(span=12, adjust=False).mean()
x["EMA_26"] = x["Close"].ewm(span=26, adjust=False).mean()
x["MACD"] = x["EMA_12"] - x["EMA_26"]
x["MACD_Signal"] = x["MACD"].ewm(span=9, adjust=False).mean()
x["MACD_Hist"] = x["MACD"] - x["MACD_Signal"]
```

8.4 RSI (14)

RSI summarizes the balance of smoothed gains and losses on a 0–100 scale. Traditional 70/30 levels are descriptive heuristics; the ML model receives the continuous RSI value and decides whether it has predictive usefulness in combination with other features.

> **Formula:** RS = Average Gain / Average Loss; RSI = 100 – 100/(1 + RS).

8.5 Bollinger Position

A 20-day mean and standard deviation define upper/lower bands. The engineered band-position feature normalizes where Close lies relative to those bands: (Close – Lower)/(Upper – Lower).

8.6 Volatility

Rolling standard deviations of daily returns over 5, 10, 20 and 60 sessions represent recent variability. Volatility is important because the same return pattern can have different meaning in quiet and turbulent regimes.

8.7 ATR, Range, Candle and Gap

True Range considers High–Low and gaps relative to previous Close. ATR-14 is its rolling mean. ATR% normalizes by price. Daily Range%, Body% and Gap% describe intraday structure without requiring raw price scale.

8.8 Volume Features

Volume Ratio compares current volume with its 20-day mean; Volume Change measures percentage change. In production, volume semantics must be checked carefully because indices and different instruments may expose volume differently.

8.9 Calendar Features

DayOfWeek and Month provide simple seasonal context. They are intentionally limited because calendar effects can be unstable and should not be assumed causal.

9. Target Engineering & Leakage Prevention

The supervised target is generated from the next close: **Target(t)=1 when Close(t+1)>Close(t), otherwise 0**. The final row has no observed next close, so it cannot become a labeled training example.

```
feat["Next_Close"] = feat["Close"].shift(-1)
feat["Target"] = np.where(
    feat["Next_Close"].notna(),
    (feat["Next_Close"] > feat["Close"]).astype(int),
    np.nan
)
```

A common incorrect implementation computes indicators using centered windows, future-filled values or randomly shuffles before time-dependent preprocessing. Those approaches leak future information. This project avoids them by using backward-looking rolling operations and chronological partitions.

10. Chronological Train / Validation / Test Design

The earliest 70% of labeled observations are training data, the next 15% are validation data and the latest 15% are test data. Random splitting is intentionally avoided. Validation is used for algorithm and threshold decisions; test remains untouched until final evaluation.

```
n = len(model_df)
train_end = int(n * 0.70)
val_end   = int(n * 0.85)

train = model_df.iloc[:train_end]
val   = model_df.iloc[train_end:val_end]
test  = model_df.iloc[val_end:]
```

11. Baseline Model

Before sophisticated algorithms, the project predicts the majority class observed in training. This gives a minimum reference. A complex model that cannot provide meaningful out-of-sample improvement over a trivial baseline is difficult to justify.

12. Logistic Regression

A linear probabilistic classifier. StandardScaler is applied inside a Pipeline. It provides a strong interpretable baseline and tests whether a roughly linear combination of engineered features contains signal.

Key parameters: max_iter=3000, class_weight='balanced'. Scaling is fit only on training data through the pipeline.

13. K-Nearest Neighbors

KNN classifies a row based on nearby historical feature vectors. Because distance is scale-sensitive, imputation and StandardScaler are mandatory parts of its pipeline.

The project uses k=15 and distance weighting. KNN can struggle in high-dimensional/noisy data, so it is included as a useful comparison rather than assumed to be best.

14. Support Vector Machine — RBF

RBF SVM can model non-linear decision boundaries. Scaling is essential. probability=True is enabled so ROC-AUC and probability-threshold analysis can be performed.

SVM may become computationally expensive as dataset size grows. C and gamma would normally be tuned using time-aware validation.

15. Random Forest

An ensemble of decision trees trained on bootstrapped samples and random feature subsets. It captures non-linear interactions and is relatively robust on tabular data.

The notebook constrains depth and leaf size to reduce overfitting and uses balanced subsampling. Hundreds of trees reduce variance.

16. Extra Trees

Extra Trees increases randomization in split selection compared with Random Forest. This can reduce variance and provides a useful ensemble benchmark.

It is fast on moderate tabular datasets and can capture interactions without feature scaling.

17. Gradient Boosting

Sequential trees are added to correct previous errors. A low learning rate with more estimators can provide smoother fitting than aggressive boosting.

Boosting can overfit noisy financial patterns, so future-period validation is essential.

18. Histogram Gradient Boosting

A histogram-based boosting method optimized for efficient tabular learning. It can capture non-linear relationships while handling larger datasets efficiently.

Regularization and depth controls are important because market data has low signal-to-noise ratio.

19. XGBoost

A widely used gradient-boosted tree library with subsampling, column sampling and regularization options. It is included as a strong tabular benchmark, not as an automatic winner.

The project uses moderate depth, low learning rate and row/column subsampling. Production tuning should be performed only with time-aware validation.

20. Evaluation Metrics

Metric	Formula / idea	Why it matters
Accuracy	Correct predictions / all predictions	Overall correctness; can hide class imbalance
Precision	TP / (TP + FP)	How often predicted UP is actually UP
Recall	TP / (TP + FN)	How many actual UP days are detected
F1	2PR / (P + R)	Balances precision and recall
ROC-AUC	Ranking quality across thresholds	Threshold-independent discrimination
Confusion Matrix	TP, TN, FP, FN counts	Shows exact error types

No single metric proves economic value. A trading research layer would additionally measure returns after costs, drawdown, turnover, hit rate, Sharpe-like risk metrics and stability by market regime.

21. Model Comparison & Threshold Selection

All candidate models are fitted on training data and evaluated on validation data. The notebook creates a comparison table and separate graphs for Accuracy, Precision, Recall, F1 and ROC-AUC. ROC curves visualize discrimination across thresholds.

The best validation model is selected using ROC-AUC and F1 ordering. Instead of automatically using probability 0.50, the notebook inspects thresholds from the validation predictions and chooses a demonstration threshold that maximizes validation F1. This threshold must never be optimized on the final test set.

```
precision, recall, thresholds = precision_recall_curve(y_val, val_prob)
f1_values = 2 * precision[:-1] * recall[:-1] / (
    precision[:-1] + recall[:-1] + 1e-12
)
best_threshold = thresholds[np.nanargmax(f1_values)]
```

22. Final Refit & Untouched Test Evaluation

After algorithm and threshold selection, training and validation observations are combined. A fresh clone of the selected pipeline is fitted on this larger historical set. Only then is the untouched latest test period evaluated. This produces the most defensible final estimate available within the project design.

The report intentionally does not invent final scores. Run the companion notebook and transfer the actual outputs into the Results Template in Section 32.

23. Feature Importance

Permutation importance is computed on the test period by shuffling one feature at a time and measuring the average reduction in ROC-AUC. A larger decrease suggests the fitted model relies more heavily on that feature for its test-period ranking performance.

Importance is not causality. Correlated predictors can share importance, and a feature may appear important only within a specific regime. Stability across folds is more informative than a single chart.

24. Walk-Forward Validation

A single test window can be unusually easy or difficult. The project therefore adds expanding-window validation: train on all history available so far, validate on the next block, expand training, then repeat. Accuracy, F1 and ROC-AUC are plotted by fold.

Industry interpretation: large variation between folds indicates regime sensitivity. A model with slightly lower average score but better stability can be operationally preferable to a model whose performance collapses in certain periods.

25. New Unseen Data Inference

The production-style interface accepts a **raw new OHLCV row**. It does not ask a user to manually calculate RSI, MACD or SMA. The new row is appended to historical data, `engineer_features()` is rerun, the latest feature vector is selected, and the final pipeline returns P(UP).

```
history_plus_new = pd.concat([history, new_raw], ignore_index=True)
new_feat = engineer_features(history_plus_new)
latest = new_feat.iloc[[-1]]

prob_up = final_model.predict_proba(
    latest[feature_cols]
)[:, 1][0]

predicted_class = int(prob_up >= best_threshold)
```

> This reduces training-serving skew because both training and inference use the same feature logic. The demonstration row in the notebook is synthetic and must be replaced with actual new OHLCV data for a real inference test.

26. Model Persistence

The final pipeline is saved using joblib. A separate JSON metadata file stores ticker, feature-column order, threshold, selected model name and target definition. Production systems should additionally store training-data version, code version, library versions, model hash and evaluation report.

```
joblib.dump(final_model, "stock_direction_model.joblib")
with open("model_metadata.json", "w") as f:
    json.dump(metadata, f, indent=2)
```

27. Deployment Architecture

Component	Responsibility
Market data job	Fetch validated EOD OHLCV
Feature service	Execute versioned feature engineering
Model artifact	Loaded sklearn/XGBoost pipeline
Inference API	FastAPI endpoint returning probability/class
Dashboard	Streamlit/web client for visualization
Prediction store	Inputs, probability, class, model version
Monitoring	Data quality, drift, latency, realized metrics
Retraining job	Train candidate and compare with champion

A production API should validate schema, reject missing required fields, log request/model versions and return both probability and metadata. It should not silently accept malformed data.

28. MLOps & Monitoring

Data monitoring: missing values, stale dates, zero/invalid prices, volume anomalies and schema changes.
Feature monitoring: distribution drift, out-of-range RSI, volatility regime changes and feature missingness.
Prediction monitoring: probability distribution, class ratio and confidence shifts.
Performance monitoring: delayed labels allow rolling Accuracy/F1/AUC after the next close becomes known.
Operational monitoring: inference latency, failures, model load errors and pipeline version.

> Retraining should not occur simply because recent performance looks poor. A documented policy should define cadence or drift triggers, candidate evaluation, approval and rollback.

29. Testing & Governance

Test	Example
Schema test	Required OHLCV columns exist
Range test	High ≥ Low; prices positive
Chronology test	Dates strictly ordered / duplicates handled
Feature test	Same historical input gives deterministic features
Leakage test	No feature uses shift(-1) or future window
Inference parity	Saved model accepts exact feature order
Regression test	Pipeline output remains within expected tolerance
Artifact test	Model and metadata versions match

30. Risks & Limitations

Stock-market prediction has an unusually low signal-to-noise ratio. Relationships can change after policy decisions, crises, earnings, geopolitical events, liquidity changes or shifts in participant behavior. Technical indicators are transformations of historical prices and do not contain all information affecting future prices.

The classification objective also differs from a profitable trading strategy. A model can correctly predict direction but still lose money if gains on correct predictions are small, losses are large, or transaction costs dominate. This project does not claim deployable profitability.

Other limitations include vendor-data quality, index-volume semantics, hyperparameters not exhaustively optimized, no probability calibration, no external macro/news variables, no transaction-cost backtest and no portfolio/risk engine.

31. Future Enhancements

Add India VIX, sector indices, benchmark-relative momentum, global-market lag variables, interest-rate/macro features and event/news features where legally and technically appropriate. Evaluate probability calibration, SHAP, Optuna with walk-forward folds, MLflow experiment tracking, DVC/data versioning, automated feature tests, drift detection and champion/challenger deployment.

For trading research, build a separate backtesting module that converts probabilities into explicitly defined positions and includes brokerage, taxes, slippage, turnover, stop/risk rules and drawdown analysis. Keep prediction-model evaluation and strategy evaluation as separate layers.

32. Results Template — Fill After Colab Execution

Field	Actual notebook result
Data date range	_____
Rows after feature engineering	_____
Majority baseline validation accuracy	_____
Best validation model	_____
Validation Accuracy	_____
Validation Precision	_____
Validation Recall	_____
Validation F1	_____
Validation ROC-AUC	_____
Selected threshold	_____
Final Test Accuracy	_____
Final Test Precision	_____
Final Test Recall	_____
Final Test F1	_____
Final Test ROC-AUC	_____
Walk-forward mean ROC-AUC	_____
Top 5 permutation features	_____
Latest new-data P(UP)	_____

Model acceptance should consider baseline improvement, metric stability across walk-forward folds, error type, drift and business/strategy requirements—not one headline accuracy number.

33. Interview / Viva Questions with Answers

1. Why did you not use random train_test_split?

Because time-series observations have temporal dependency. Random shuffling lets later regimes influence training while earlier rows appear in test, producing an unrealistic deployment simulation.

2. What is data leakage here?

Any predictor containing information unavailable at prediction time—for example next-day Close, future-filled values or future observations inside rolling calculations.

3. Why use multiple algorithms?

Different models make different assumptions. Comparing linear, distance, kernel, bagging and boosting methods helps determine whether signal is robust to model choice.

4. Why scale Logistic Regression, KNN and SVM?

Their optimization or distance/kernel calculations are affected by feature magnitude. Tree models generally do not require the same scaling.

5. Why ROC-AUC and F1 along with accuracy?

Accuracy alone can hide class imbalance and error trade-offs. F1 balances precision/recall; ROC-AUC measures ranking quality across thresholds.

6. Why choose threshold on validation only?

Choosing it on test would leak test information into model decisions and make final performance optimistic.

7. Why walk-forward validation?

It tests the model across multiple future periods and exposes regime sensitivity.

8. Why permutation importance?

It works with any fitted model and measures how much predictive performance deteriorates when a feature is disrupted.

9. How do you predict new data?

Append raw OHLCV to sufficient history, rerun identical feature engineering, select the latest feature row and call the saved pipeline's predict_proba.

10. Is this a trading system?

No. It is a direction-classification ML project. A trading system additionally requires execution rules, transaction costs, risk management and backtesting.

34. Conclusion

The project demonstrates a complete machine-learning lifecycle rather than only fitting a classifier. It begins with a precise business/ML question, validates data, visualizes market behavior, engineers backward-looking features, prevents leakage, respects chronological validation, benchmarks eight algorithms, separates validation decisions from final testing, inspects feature reliance and temporal stability, supports new raw-data inference, persists artifacts and defines a path to monitored deployment.

The most important lesson is that in financial ML, **methodology and leakage control matter more than an impressive training score**. A credible project must show what information was available at prediction time, how future periods were protected, how stability was tested and what limitations remain.

Educational use only — not investment advice.